Vizuara Technologies

Build LLM from Scratch: Research-Based Assessment

Training a GPT-Style Language Model from Scratch:

Effects of Training Duration, Learning Rate, Depth, Attention Heads and Block Components

---

Pratik Ramdas Sonawane

Vizuara Technologies

Code and results: Google Colab notebook

October 2026

Abstract

We train a scaled-down GPT-2-style decoder-only transformer (28.95M parameters) from scratch on Jane Austen's *Pride and Prejudice* (168k training tokens) and study how training duration, learning rate, depth, the number of attention heads, and individual block components affect training and validation loss. All 47 runs share one protocol (seeded initialisation and data order, AdamW, constant learning rate, gradient clipping) and are evaluated after every epoch on held-out final chapters of the book. Every configuration overfits the single-book corpus: the lowest validation loss (≈ 4.61, perplexity ≈ 100) is reached within 10–30 epochs, after which training loss approaches zero while validation loss rises towards 10. In this data-limited regime, architecture has little influence. Depth (1–12 layers) changes the best validation loss by less than 0.03 with no consistent trend across three seeds, and fewer, wider attention heads are slightly but consistently better (4.663 with one head vs. 4.682 with eight). Ablations show that residual connections are indispensable: without them the model does not progress beyond a unigram (word-frequency) predictor (6.33). Removing the feed-forward network has no measurable effect, and removing LayerNorm *improves* the best validation loss by 0.18–0.21, which we attribute to gradient clipping supplying the stability that normalisation usually provides. Finally, we show that a fixed-epoch, single-seed protocol produced misleading ablation conclusions, which were corrected by comparing models at their individual best epochs and by repeating experiments over several seeds.

Summary of Key Findings

Setup. A 4-layer, 256-dimensional GPT (28.95M parameters, 89% of them in the embedding and output layers) trained from scratch on one novel (168k training tokens; validation = the last 10% of the book). 47 runs, about 4.2 GPU-hours on a Colab T4. Every run uses the same seed, data order, optimiser (AdamW), constant learning rate and gradient clipping, unless the experiment changes it.

1. **Training duration matters more than architecture at this data scale.** Every model reaches its lowest validation loss within 10–30 epochs. With more epochs it memorises the book: training loss falls to 0.006 while validation loss climbs back to ≈ 10, close to random guessing ($\ln 50257 = 10.82$).

2. **A higher learning rate overfits sooner.** The best epoch is 25 at 10^{-4}, 14 at 10^{-3} and 53 at 10^{-2}, the last after a slow, noisy start; no run diverged. The best full-model result in the project (4.616) came from the intermediate rate 4×10^{-4} used for the baseline.

3. **Depth has no consistent effect.** Over three seeds, 1 to 12 layers give 4.652–4.679 with no monotonic trend, while the cost per epoch grows by about 45%.

4. **Fewer, wider heads are slightly better.** With the width fixed at 256, one head (4.663 $\pm$ 0.002) beats eight heads (4.682 $\pm$ 0.007). The effect is consistent across seeds but small (perplexity 106 vs. 108).

5. **Residual connections are essential.** Without them, both the 4- and the 12-layer model stay at 6.33 for 60 epochs, which is exactly the loss of a unigram word-frequency model (6.32), and their gradient norm decays.

6. **LayerNorm is not needed at this scale.** Removing it lowers the best validation loss (4.642 vs. 4.818 with 4 layers; 4.633 vs. 4.841 with 12 layers, LR 10^{-3}). Its gradient norm is higher, but gradient clipping at 1.0 keeps training stable.

7. **The feed-forward network gives no measurable benefit with one book.** Once both models train to their minimum, the full model and the model without an FFN are within 0.03 of each other.

8. **The data, not the architecture, sets the limit.** The best transformer (4.61) beats a simple bigram counting model (4.94) by only 0.33. Capacity-adding components (depth, FFN, more heads) cannot help without enough data to fill them; trainability components (residual connections) are needed regardless.

9. **Method lesson.** Comparing models after a fixed number of epochs with a single seed suggested that the FFN helps and cast doubt on the LayerNorm result. Comparing every model at its own best epoch and repeating over seeds corrected both conclusions.

Contents

Summary of Key Findings ii

1 Introduction 1

2 Background 2

2.1 Language modelling objective . . . 2
2.2 The pre-norm transformer block . . . 2
2.3 Expected roles of the components . . . 2

3 Experimental Setup 2

3.1 Data . . . 2
3.2 Model . . . 3
3.3 Training and evaluation protocol . . . 4
3.4 Count-based reference models . . . 4
3.5 Reproducibility and sanity checks . . . 5

4 Baseline 5

5 Experiment 1: Epochs and Learning Rate 6

6 Experiment 2: Number of Transformer Layers 7

7 Experiment 3: Number of Attention Heads 8

8 Experiment 4: Ablation Studies 9

8.1 Initial protocol: LR 10^{-4}, 38 epochs . . . 9
8.2 Revised protocol: LR 10^{-3}, 60 epochs . . . 10
8.3 Inference quality . . . 12
8.4 Answer to RQ4: the role of each component . . . 12

9 Discussion 13

10 Limitations and Future Work 15

11 Conclusion 15

A Code 17

B Additional Figures 18

C Complete Results 20

1 Introduction

Large language models (LLMs) are built from a small set of components that are repeated many times: token and position embeddings, multi-head self-attention, position-wise feed-forward networks, layer normalisation and residual connections [1, 2]. In large-scale models each component has a well-documented role, but how much each one matters depends on the training regime: the amount of data, the model size, and optimisation settings such as the learning rate. This report studies these design decisions empirically by training a small GPT-style model from scratch and changing one factor at a time.

We use a single public-domain novel as the training corpus. This keeps every experiment affordable on a free GPU, but it also places the model in a *data-limited* regime: the model has far more parameters than training tokens. A central theme of the results is therefore how architecture choices behave when data, rather than capacity, is the binding constraint.

We address four questions:

RQ1. How do the number of training epochs and the learning rate affect training and validation loss?

RQ2. How does the number of transformer layers affect learning?

RQ3. How does the number of attention heads affect learning when the model width is fixed?

RQ4. What is the role of LayerNorm, residual (shortcut) connections and the feed-forward network, as revealed by removing each of them?

Contributions.

- A controlled study of 47 training runs under one fixed protocol, with per-epoch training and validation loss, gradient norms and generated text for every run.
- Two count-based reference models (unigram and bigram) that make the loss values interpretable, and that identify exactly what a model without residual connections learns.
- A follow-up phase that re-runs the ablations at a higher learning rate until every variant reaches its best epoch, and repeats the depth and head experiments over three seeds, separating real effects from seed-to-seed variation.
- An analysis that distinguishes *capacity* components (depth, FFN, heads), whose benefit depends on data, from *trainability* components (residual connections, normalisation), whose benefit depends on depth and optimisation.

The complete code, together with the outputs of every run, is available as a Google Colab notebook.[1]

Section 2 reviews the architecture, Section 3 describes data, model and protocol, Sections 4 to 8 report the experiments, Section 9 discusses the results, and Sections 10 and 11 cover limitations and conclusions.

[1] https://colab.research.google.com/drive/1zfz-atgfb0Z72t7IwxTM5HA-_C3_LFRG?usp=sharing

2 Background

2.1 Language modelling objective

Given a sequence of tokens $x_1, \ldots, x_T$, a decoder-only language model predicts each next token from the preceding ones. Training minimises the average cross-entropy

$$\mathcal{L} = -\frac{1}{T}\sum_{t=1}^{T} \log p_\theta(x_{t+1} \mid x_{\le t}), \tag{1}$$

and we report perplexity $\mathrm{PPL} = e^{\mathcal{L}}$, the effective number of equally likely choices per token. A model that assigns uniform probability over the vocabulary V has $\mathcal{L} = \ln|V|$; for the GPT-2 tokenizer ($|V| = 50{,}257$) this is 10.82.

2.2 The pre-norm transformer block

Each block of a GPT-2-style model [2] applies two sub-layers, each wrapped in a residual connection [4] and preceded by layer normalisation [5, 6]:

$$h = x + \mathrm{Dropout}\big(\mathrm{MHA}(\mathrm{LN}_1(x))\big), \tag{2}$$

$$y = h + \mathrm{Dropout}\big(\mathrm{FFN}(\mathrm{LN}_2(h))\big). \tag{3}$$

Causal multi-head attention splits the model width d into H heads of size $d_h = d/H$. Each head computes

$$\mathrm{Attn}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_h}} + M\right)V, \tag{4}$$

where M masks future positions. The feed-forward network is applied to each position independently: $\mathrm{FFN}(z) = W_2\,\mathrm{GELU}(W_1 z + b_1) + b_2$ with hidden size $4d$ [7]. Figure 1 shows the block and the three ablation switches used in Experiment 4.

2.3 Expected roles of the components

- **Residual connections** give gradients a direct path to early layers and let each block learn a small correction to its input; without them deep networks suffer from vanishing gradients [4].
- **LayerNorm** keeps activations at a stable scale, which allows larger learning rates and deeper stacks [5]; the pre-norm placement is known to stabilise transformer training [6].
- **The FFN** holds about two thirds of each block's parameters and has been shown to act as a key-value memory that stores much of a model's knowledge [8].
- **Multiple heads** let a layer attend to several patterns at once, although many heads can be pruned in trained models with little loss [9].
- **Scale.** Loss improves predictably with model size and data, and compute-optimal training uses roughly 20 tokens per parameter [10, 11]. Our setting is far from this: about 0.006 tokens per parameter.

3 Experimental Setup

3.1 Data

The corpus is *Pride and Prejudice* (Project Gutenberg, book 1342). We remove the Gutenberg licence header and footer, the `[Illustration]` captions and the indentation used for centred

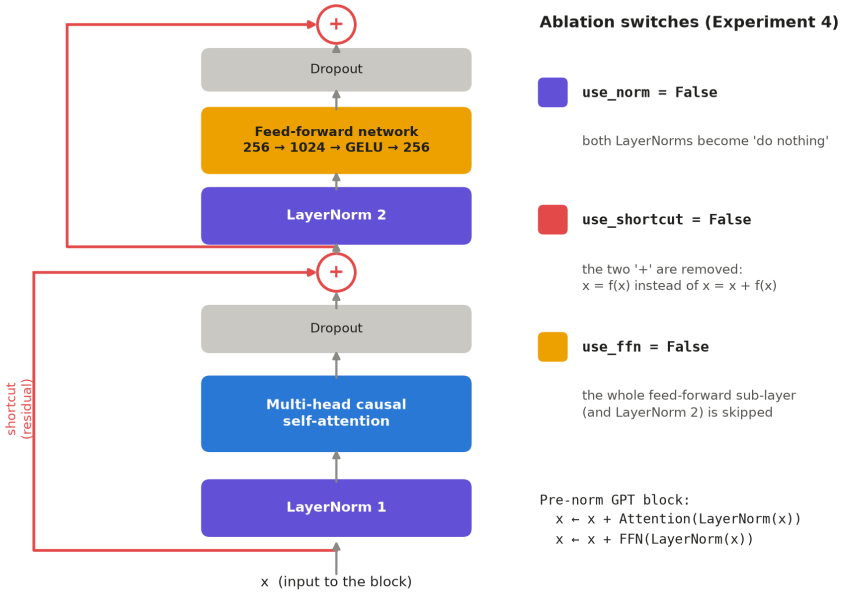


Figure 1: The pre-norm transformer block used in this study and the three ablation switches. Each switch removes the component from *every* block; the final LayerNorm before the output head is always kept.

titles, and collapse repeated blank lines. The text is tokenised with the GPT-2 byte-pair encoding [14, 2]. The first 90% of tokens form the training set and the last 10% the validation set, so validation consists of complete final chapters that never appear in training. Both sets are cut into non-overlapping windows of 256 tokens; the target of each window is the input shifted by one token. Table 1 summarises the data.

Table 1: Dataset statistics.

Quantity	Value
Characters after cleaning	718,925
Total tokens (GPT-2 BPE)	186,785
Training tokens (first 90%)	168,106
Validation tokens (last 10%)	18,679
Context length	256
Training / validation sequences	656 / 72
Batch size / steps per epoch	16 / 41

3.2 Model

The model follows GPT-2 [2, 3] but is scaled down so that the study fits on a free GPU (Table 2). With the base configuration it has 28,953,600 parameters. Of these, 25.7M (89%) belong to the token embedding and the output head ($2 \times 50{,}257 \times 256$); each transformer block has 788,992 parameters, of which 525,568 (66.6%) are in the FFN and 262,400 in attention. Changing the depth from 1 to 12 layers therefore changes the total only from 26.6M to 35.3M, and removing the FFN saves 2.1M parameters with 4 layers and 6.3M with 12. The number of heads does not change the parameter count.

Table 2: Base model configuration. Experiments 2–4 change one entry at a time.

Setting	Value	Note
Vocabulary size	50,257	GPT-2 BPE tokenizer
Context length	256	GPT-2: 1,024
Embedding dimension d	256	GPT-2: 768
Attention heads H	4	varied in Experiment 3
Transformer layers	4	varied in Experiment 2
Dropout [15]	0.1	embeddings, attention, residual branches
QKV bias	no	as in GPT-2
Initialisation	$\mathcal{N}(0, 0.02^2)$, zero biases	GPT-2 scheme

3.3 Training and evaluation protocol

Every run is produced by a single training function, so runs differ only in the factor under study (Table 3). We use AdamW [12] with a *constant* learning rate and clip the global gradient norm at 1.0 [13]. After every epoch we measure the training loss on a fixed random subset of 160 training sequences and the validation loss on all 72 validation sequences, both in evaluation mode (dropout off), and we record the mean gradient norm before clipping.

Because every configuration eventually overfits, the final loss depends mainly on how long a model was trained. Our primary metric is therefore the **best validation loss** and the epoch at which it occurs (the *best epoch*), i.e. the result early stopping would select.

Table 3: Training and evaluation protocol shared by all runs.

Item	Setting
Optimiser	AdamW, $\beta = (0.9, 0.999)$, weight decay 0.1
Learning-rate schedule	constant (no warm-up, no decay)
Gradient clipping	global norm 1.0
Batch size	16 sequences of 256 tokens (41 steps per epoch)
Precision	fp16 mixed precision with gradient scaling
Seed	123 (Experiments 2–3 additionally 42 and 7)
Evaluation	every epoch: train loss (160 fixed sequences), validation loss (all 72)
Text samples	prompt “I am sure that”, 50 tokens, temperature 0.8, top-k 40, seed 123
Hardware	NVIDIA T4 (Google Colab)

3.4 Count-based reference models

To interpret loss values, we evaluate two models that only count tokens in the training set and need no training:

- a **unigram** model, $p(x_{t+1}) = (c(x_{t+1}) + \alpha)/(N + \alpha|V|)$ with $\alpha = 0.1$, which knows only how frequent each token is; and
- an interpolated **bigram** model, $p(x_{t+1} \mid x_t) = \lambda\, c(x_t, x_{t+1})/c(x_t) + (1 - \lambda)\, p_{\text{uni}}(x_{t+1})$ with $\lambda = 0.7$, which conditions on the previous token only.

On the validation set they reach **6.32** and **4.94**. The value of λ was chosen on the validation set from $\{0.3, \ldots, 0.8\}$, so 4.94 is a slightly optimistic reference. Table 4 places these values on one scale together with our main results.

Table 4: Reference points for interpreting the loss (validation set, nats per token).

Model	Loss	PPL	What it captures
Uniform guessing	10.82	50,257	nothing
Unigram counts	6.32	556	token frequency only
Bigram counts	4.94	140	previous token only
Best transformer (this work)	4.61	101	up to 256 tokens of context

3.5 Reproducibility and sanity checks

A fixed seed (123) controls the initial weights, the shuffling of training sequences, dropout masks and text sampling, so repeated runs give the same result up to small non-determinism in GPU kernels. Each finished run is stored as a JSON file named after its full configuration, and is reloaded rather than retrained when the same configuration appears again. Long runs are checkpointed every 25 epochs. Before the experiments we verified that (i) the untrained model's loss is close to $\ln |V| = 10.82$ (observed: 10.859), (ii) the model can drive the loss on a single batch close to zero (0.042 after 200 steps), and (iii) all ablation variants produce finite logits of the expected shape. The full study comprises 47 runs and about 4.2 GPU-hours. The notebook, including all training logs, figures and tables, can be opened and re-run at this Colab link.

4 Baseline

The baseline uses the base configuration, learning rate 4×10^{-4} and 20 epochs (Fig. 2). Training and validation loss fall together for the first few epochs. The best validation loss, **4.616** (perplexity 101), is reached at **epoch 10**. After that the training loss keeps falling (to 2.47 at epoch 20) while the validation loss rises to 4.97: the model has started to overfit. Training took 1.4 minutes. This learning rate later turned out to give the lowest validation loss of any full model in the study.

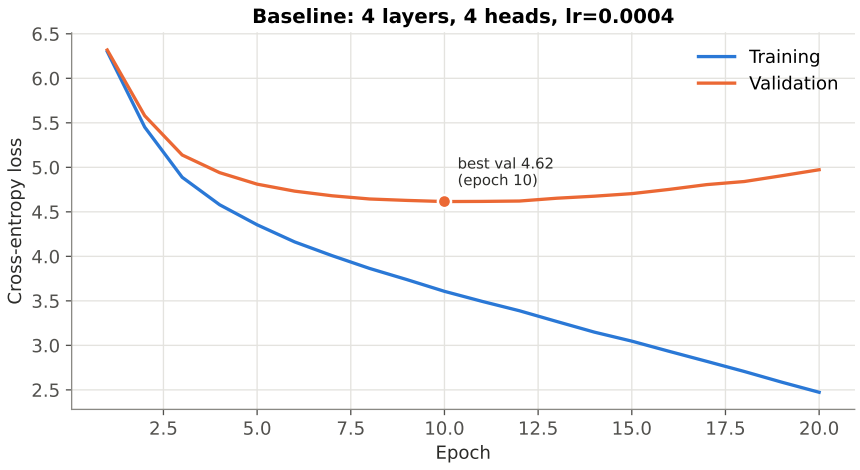


Figure 2: Baseline (4 layers, 4 heads, LR 4×10^{-4}). The dot marks the best validation loss.

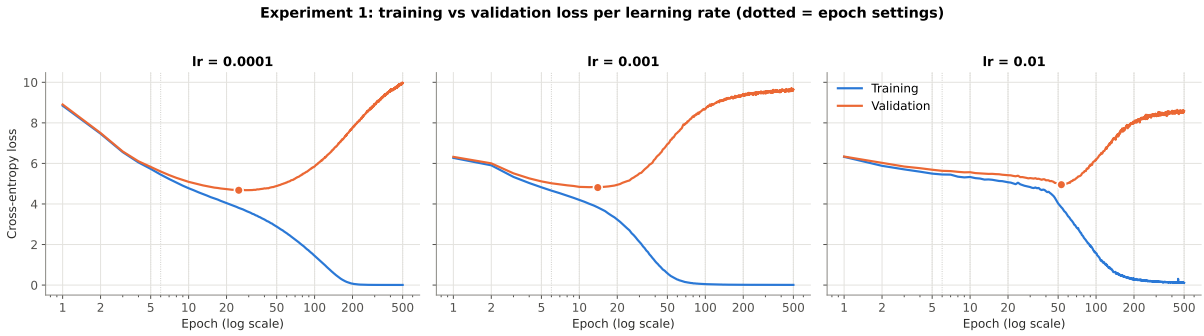


Figure 3: Training and validation loss for each learning rate over 500 epochs (logarithmic epoch axis). Dots mark the best validation loss; dotted lines mark the evaluated epoch settings.

Table 5: Loss after N epochs for each learning rate. Bold: lowest validation loss among the evaluated epoch settings.

	Epochs						
LR	5	10	20	50	100	200	500
Validation loss							
1×10^{-4}	5.83	5.09	**4.73**	4.88	5.85	7.77	9.97
1×10^{-3}	5.11	**4.84**	4.95	6.95	8.71	9.38	9.64
1×10^{-2}	5.69	5.57	5.42	**4.99**	6.17	8.00	8.59
Training loss							
1×10^{-4}	5.73	4.78	4.05	2.88	1.45	0.07	0.01
1×10^{-3}	4.82	4.20	3.23	0.59	0.05	0.01	0.01
1×10^{-2}	5.50	5.33	5.07	4.03	1.57	0.27	0.11

5 Experiment 1: Epochs and Learning Rate

Design. We test learning rates of 10^{-4}, 10^{-3} and 10^{-2} and read off the losses after 5, 10, 20, 50, 100, 200 and 500 epochs. Because the learning rate is constant and the seed is fixed, a 500-epoch run passes through every shorter setting exactly as a separate run would. We therefore train one 500-epoch run per learning rate (3 runs instead of 21). A separate 20-epoch run at 10^{-3} confirmed this: its validation curve matches the first 20 epochs of the long run to within 0.02. The remaining difference comes from non-deterministic GPU arithmetic accumulating over 20 epochs. It is of the same order as the seed-to-seed variation reported in Sections 6 and 7, which is a further reason to treat differences below about 0.02 as noise.

Results. Figure 3 and Tables 5 and 6 show the same pattern for every learning rate:

- **U-shaped validation loss.** Validation loss falls, reaches a minimum, then rises steadily. The training loss falls monotonically to 0.006 (10^{-4}) and 0.009 (10^{-3}): the model memorises the training chapters.
- **Extreme overfitting with long training.** After 500 epochs the validation loss is 9.97, 9.65 and 8.59, close to uniform guessing (10.82). A memorised model is highly confident, so its errors on unseen text are penalised heavily by the cross-entropy. The generalisation gap grows accordingly (Fig. 4).
- **A higher learning rate overfits sooner.** The best epoch moves from 25 (10^{-4}) to 14

Table 6: Experiment 1 summary (500 epochs per learning rate).

LR	Best val.	Best epoch	PPL	Train @500	Val. @500	Minutes
1×10^{-4}	4.676	25	107.3	0.006	9.973	36.2
1×10^{-3}	4.809	14	122.7	0.009	9.645	35.8
1×10^{-2}	4.951	53	141.4	0.113	8.588	35.7

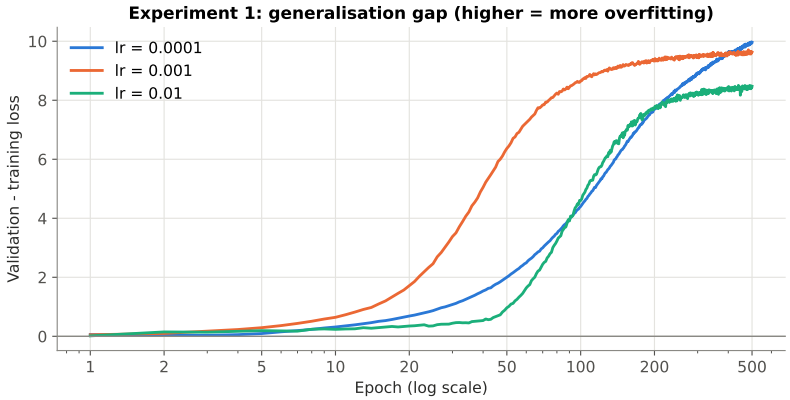


Figure 4: Generalisation gap (validation minus training loss) over training.

(10^{-3}).

- 10^{-2} **is noisy but stable.** With AdamW and gradient clipping no run diverged, but at 10^{-2} the validation loss improved only slowly (5.69 at epoch 5, 4.99 at epoch 50). Its minimum (4.951 at epoch 53) was followed by a sudden drop in training loss as memorisation began.
- **The smaller learning rate reached the lower minimum** (4.676 vs. 4.809 vs. 4.951). The baseline rate 4×10^{-4} was better still (4.616), so the optimum lies between 10^{-4} and 10^{-3} and the tested grid is coarse.

Answer to RQ1. On this corpus the useful training horizon is short: 10–25 epochs depending on the learning rate. Additional epochs only add memorisation. The learning rate controls both how fast the model learns and how soon it begins to overfit.

Settings for Experiments 2–4. Architecture experiments require a fixed optimisation setting. The notebook selected the learning rate with the lowest best validation loss in Experiment 1 (10^{-4}) and $\lceil 1.5 \times$ its best epoch$\rceil = 38$ epochs, so that the curves also show the onset of overfitting. Section 8 shows that this low learning rate and fixed budget bias some comparisons; the ablations were therefore repeated under a revised protocol.

6 Experiment 2: Number of Transformer Layers

We train models with 1, 3, 5, 7 and 12 layers (4 heads, LR 10^{-4}, 38 epochs). Because the differences in the first run were small, every depth was repeated with seeds 42 and 7.

Results. The training and validation curves for all depths are nearly identical (Fig. 5); even the training loss after 38 epochs differs by only 0.06 (3.31–3.37). Over three seeds (Fig. 6 and Table 7) the best validation loss ranges from 4.652 (12 layers) to 4.679 (5 layers) without

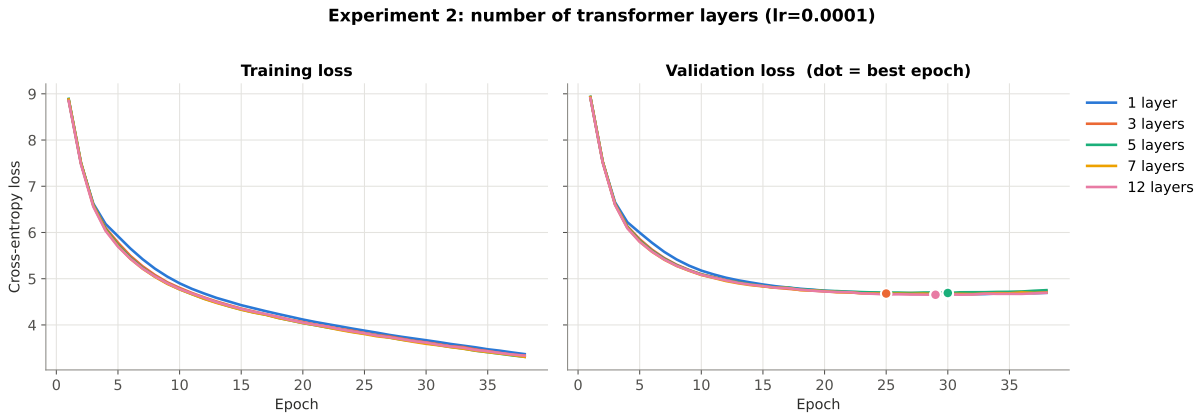


Figure 5: Training and validation loss for different depths (seed 123). The curves almost coincide.

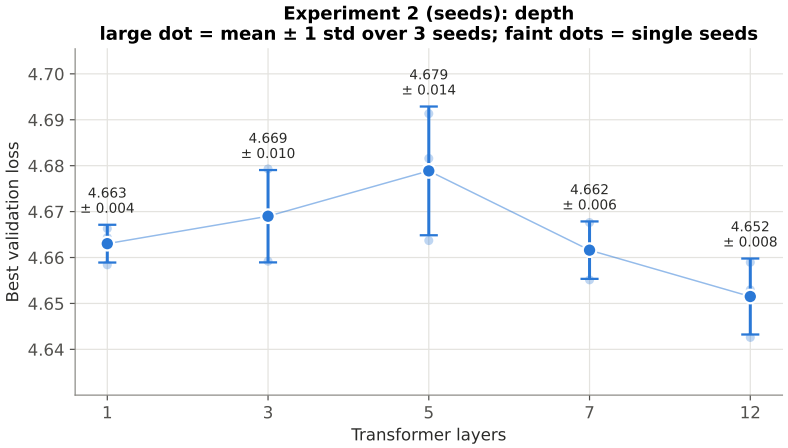


Figure 6: Best validation loss against depth over three seeds (mean ± one standard deviation; faint dots are individual seeds).

a monotonic trend: it rises from 1 to 5 layers and falls again at 7 and 12. The seed-to-seed standard deviation is 0.004–0.014. The difference between the best and worst depth (0.027) is about three times the average standard deviation, but most error bars overlap and only 12 versus 5 layers separate clearly. Meanwhile, the time per epoch grows by about 45% from 1 to 12 layers.

Answer to RQ2. On this dataset depth has no consistent effect. Extra layers add capacity, but the models memorise the corpus before that capacity can improve generalisation (Section 9).

7 Experiment 3: Number of Attention Heads

We train models with 1, 2, 4 and 8 heads at a fixed width of 256, so the head dimension is 256, 128, 64 and 32. The parameter count is identical for all four models, so this experiment changes only how attention capacity is divided.

Results. The best validation loss increases monotonically with the number of heads: 4.663 (1 head), 4.666 (2), 4.678 (4) and 4.682 (8) (Fig. 8 and Table 8). The seed-to-seed standard deviation is very small (0.002–0.007), and the 1-versus-8 difference (0.018) is about five times

Table 7: Experiment 2: depth. PPL is the perplexity of the three-seed mean.

		Seed 123		3 seeds (123, 42, 7)		
Layers	Params (M)	Best val.	Epoch	Mean ± std	PPL	Min./run
1	26.59	4.664	30	4.663 ± 0.004	106.0	2.4
3	28.16	4.679	25	4.669 ± 0.010	106.6	2.6
5	29.74	4.691	30	4.679 ± 0.014	107.7	2.8
7	31.32	4.668	29	4.662 ± 0.006	105.8	3.0
12	35.27	4.653	29	4.652 ± 0.008	104.7	3.5

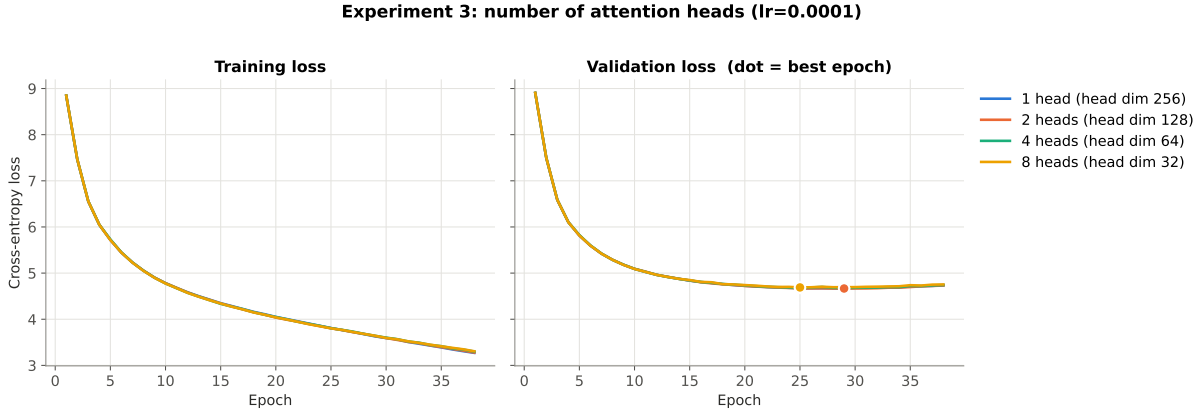


Figure 7: Training and validation loss for different numbers of heads (seed 123).

the average standard deviation, so the ordering is reliable. The effect itself is small, about 2% in perplexity (106 vs. 108).

Answer to RQ3. With the width fixed, fewer and wider heads are slightly better here. Splitting 256 dimensions into eight 32-dimensional heads makes each query-key comparison lower-dimensional, while a single book appears to require only a few distinct attention patterns. This differs from the usual large-scale setting, where many heads are standard, and is consistent with observations that many heads are redundant [9].

8 Experiment 4: Ablation Studies

Each ablation removes one component from every transformer block (Fig. 1): both LayerNorms (`use_norm=False`), both residual additions (`use_shortcut=False`, so $y = f(x)$), or the FFN sub-layer (`use_ffn=False`). Ablations are run with 4 layers and, because residual connections and normalisation are expected to matter more in deep networks, also with 12 layers.

8.1 Initial protocol: LR 10^{-4}, 38 epochs

Under the setting chosen in Experiment 1 (Fig. 9 and Table 9), removing the shortcut clearly hurts (5.564 with 4 layers, 6.316 with 12), removing the FFN appears to hurt (4.747 vs. 4.676; 4.824 vs. 4.653), and removing LayerNorm appears to help slightly (4.627; 4.613). The *best epoch* column, however, reveals a problem. The full model peaks at epochs 25–29 and then overfits, whereas the variants without LayerNorm or without the FFN peak at epoch 37 of 38: they were still improving when training stopped. The comparison therefore mixes a model past

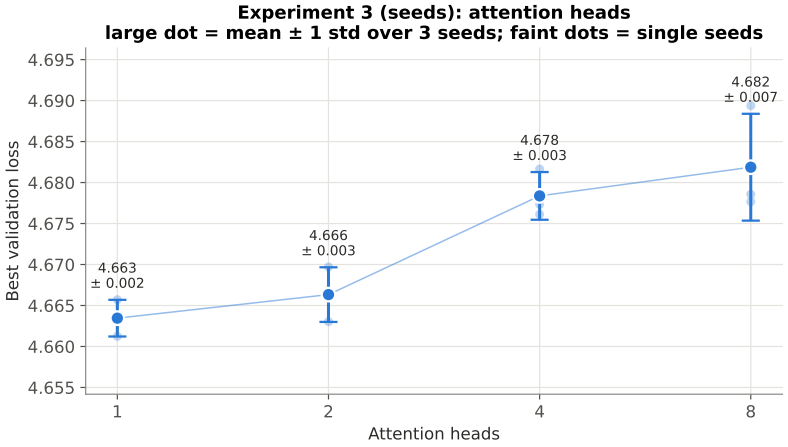


Figure 8: Best validation loss against the number of heads over three seeds.

Table 8: Experiment 3: attention heads at fixed width $d = 256$.

		Seed 123		3 seeds (123, 42, 7)	
Heads	Head dim.	Best val.	Epoch	Mean ± std	PPL
1	256	4.666	29	4.663 ± 0.002	106.0
2	128	4.666	29	4.666 ± 0.003	106.3
4	64	4.676	25	4.678 ± 0.003	107.6
8	32	4.689	25	4.682 ± 0.006	108.0

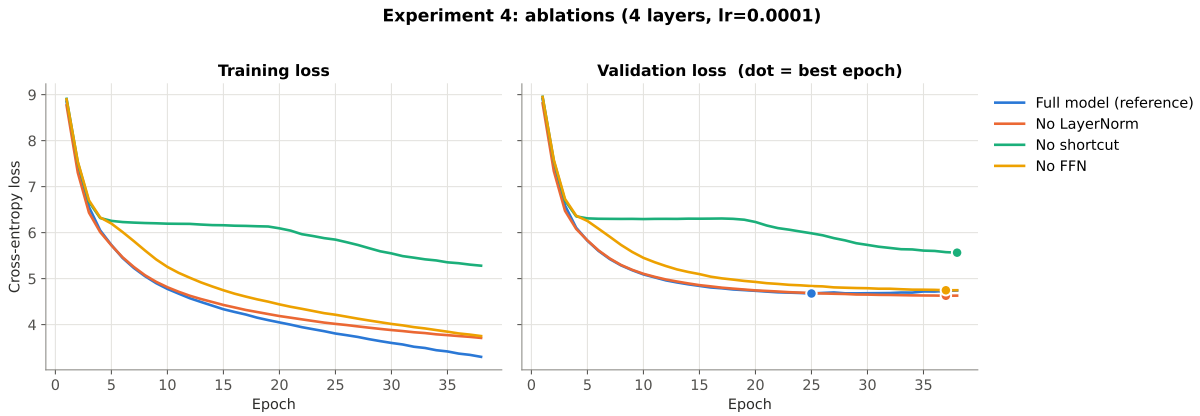
its optimum with models that have not reached theirs. In addition, the low learning rate makes optimisation easy, which may hide the stabilising role of LayerNorm. We therefore repeated the ablations under a revised protocol.

8.2 Revised protocol: LR 10^{-3}, 60 epochs

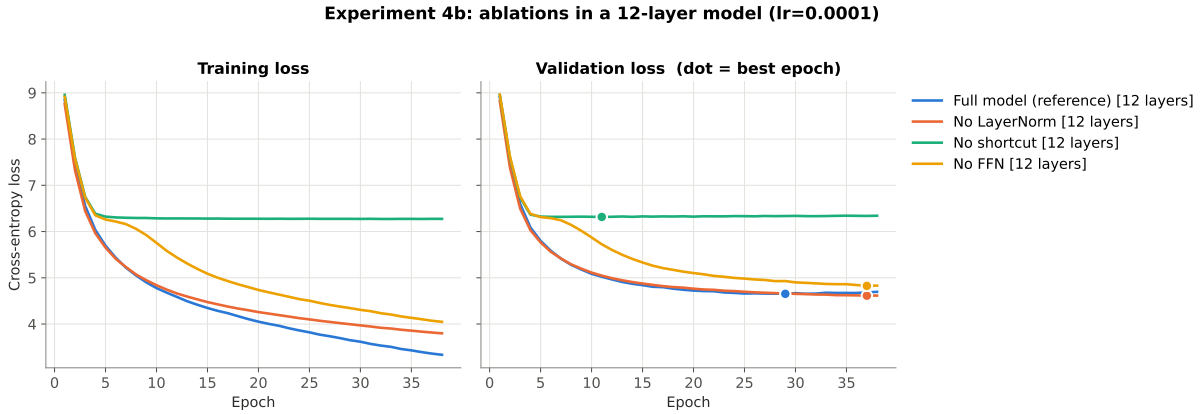
All four variants were retrained with 4 and 12 layers at a ten times higher learning rate (10^{-3}) and for 60 epochs. Every run reached its minimum by epoch 20 at the latest, well before the end of training, so each model is now compared at its own best epoch (Table 10).

Without residual connections the model learns only token frequencies. At 10^{-3} both the 4- and the 12-layer model reach their best validation loss at epoch 2 (6.329 and 6.333) and stay there for the remaining 58 epochs; the training loss also stays at 6.27 (Fig. 10). This value matches the unigram model of Section 3.4 (6.32): the network learns the frequency of each token and nothing about context. Its gradient norm decreases from about 0.6 to 0.3 over training (Fig. 11), the signature of vanishing gradients when each layer must transform its input completely rather than add a correction. At 10^{-4} the 4-layer variant slowly escaped this plateau (5.564); at 10^{-3} neither depth did.

Removing LayerNorm improves the best validation loss. The variant without LayerNorm is better than the full model at both depths and both learning rates: 4.642 vs. 4.818 (4 layers) and 4.633 vs. 4.841 (12 layers) at 10^{-3}. Every run converged, and the gap of 0.18–0.21 is about twenty times the seed-to-seed standard deviation measured in Experiments 2–3, so this is not an artefact of the stopping point. Three observations explain it:



(a) 4 layers



(b) 12 layers

Figure 9: Ablations under the initial protocol (LR 10^{-4}, 38 epochs).

1. *Gradient clipping substitutes for normalisation.* Without LayerNorm the gradient norm is higher: about 2.4 versus at most 1.2 for the full model with 4 layers, and 0.9–1.6 versus 0.5–1.15 with 12 layers (Fig. 11). This is the instability that LayerNorm normally prevents, but clipping at 1.0 bounds every update, so training never diverged.
2. *Faster early learning and an earlier optimum.* Without LayerNorm the validation loss after the first epoch is already about 5.7, versus 6.3 for the full model, and the minimum is reached at epoch 5 instead of 12 (4 layers). One plausible explanation, which we did not test directly, involves the initialisation: with $\mathcal{N}(0, 0.02^2)$ weights and no normalisation, the block outputs are initially small, so the network first behaves like a shallow model close to a bigram predictor, which generalises comparatively well on little data. With pre-norm, each block receives unit-scale inputs from the first step and its full capacity becomes available immediately.
3. *Normalisation is not removed entirely.* The final LayerNorm before the output head is retained, which keeps the logits well scaled.

Removing the FFN has no measurable effect. Once both models are compared at their best epoch, the variant without FFN equals the full model: 4.823 vs. 4.818 (4 layers) and 4.816 vs. 4.841 (12 layers), differences within seed variation. The apparent advantage of the FFN under the initial protocol (+0.07 and +0.17) was an artefact of stopping the slower-learning variant before its minimum. After its minimum, the model without FFN overfits most strongly of all variants with 4 layers (final validation loss 8.25).

Table 9: Ablations under the initial protocol (LR 10^{-4}, 38 epochs). Upper block: 4 layers; lower block: 12 layers.

Variant	Layers	Params (M)	Best val.	Best epoch	Final train	Final val.
Full model (reference)	4	28.95	4.676	25	3.299	4.741
No LayerNorm	4	28.95	4.627	37	3.710	4.630
No shortcut	4	28.95	5.564	38	5.281	5.564
No FFN	4	26.85	4.747	37	3.751	4.748
Full model (reference)	12	35.27	4.653	29	3.335	4.696
No LayerNorm	12	35.25	4.613	37	3.799	4.616
No shortcut	12	35.27	6.316	11	6.274	6.342
No FFN	12	28.95	4.824	37	4.047	4.831

Table 10: Best validation loss and best epoch under both protocols, plus the final losses of the revised protocol.

		LR 10^{-4}, 38 epochs		LR 10^{-3}, 60 epochs			
L	Variant	Best val.	Epoch	Best val.	Epoch	Final train	Final val.
4	Full model (reference)	4.676	25/38	4.818	12/60	0.273	7.454
4	No LayerNorm	4.627	37/38	4.642	5/60	0.403	7.476
4	No shortcut	5.564	38/38	6.329	2/60	6.269	6.364
4	No FFN	4.747	37/38	4.823	10/60	0.123	8.250
12	Full model (reference)	4.653	29/38	4.841	20/60	0.320	7.171
12	No LayerNorm	4.613	37/38	4.633	10/60	0.233	7.375
12	No shortcut	6.316	11/38	6.333	2/60	6.269	6.363
12	No FFN	4.824	37/38	4.816	16/60	0.536	7.340

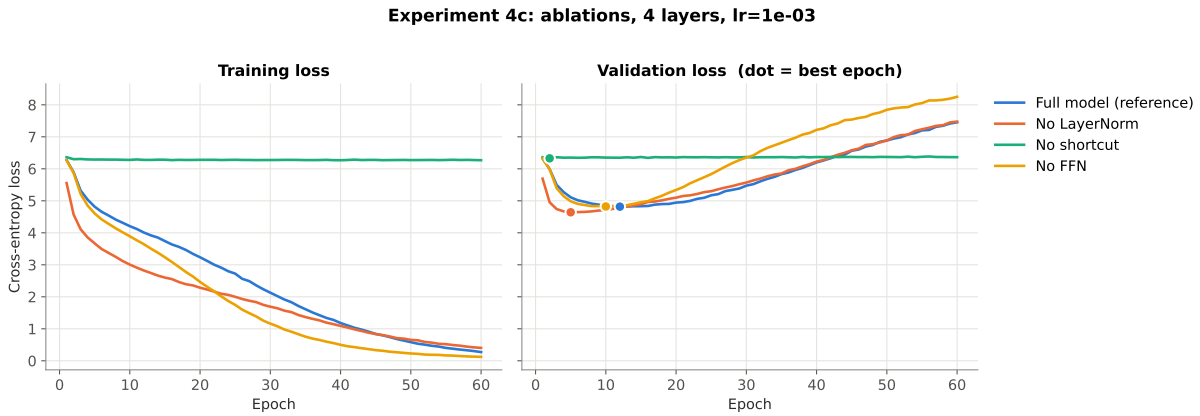
The higher learning rate itself. The full model is worse at 10^{-3} than at 10^{-4} (4.818 vs. 4.676 with 4 layers; 4.841 vs. 4.653 with 12), in line with Experiment 1: faster optimisation leads to earlier and stronger memorisation of the small corpus. After 60 epochs every variant except the one without shortcuts has a training loss below 0.6 and a validation loss above 7.

8.3 Inference quality

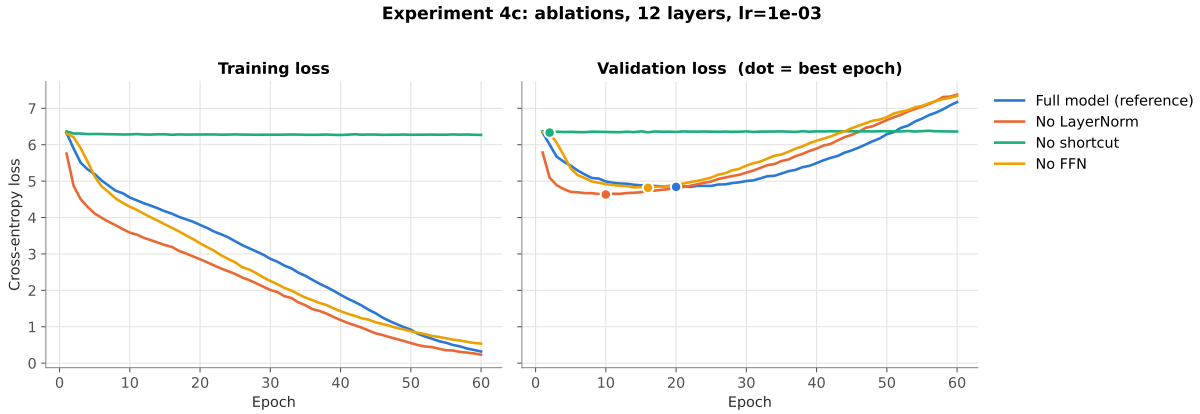
Table 11 shows text generated from the same prompt and sampling seed. Early in training the model produces frequent tokens and punctuation; by epoch 10 it produces grammatical fragments with characters and dialogue from the novel. Heavily trained models (epoch 60 at 10^{-3}, epoch 500) write fluent, Austen-like sentences that are partly recombined from memorised passages. The model without residual connections produces only high-frequency tokens ("as as the„."), which matches its unigram-level loss. The variants without LayerNorm and without FFN produce text of similar quality to the full model, consistent with their losses.

8.4 Answer to RQ4: the role of each component

- **Residual connections** are what make the network trainable. Without them, even a 4-layer model cannot learn more than token frequencies.
- **LayerNorm** provides stability, but at this depth, with gradient clipping and learning rates up to 10^{-3}, that stability is not needed; removing it gave better generalisation.



(a) 4 layers



(b) 12 layers

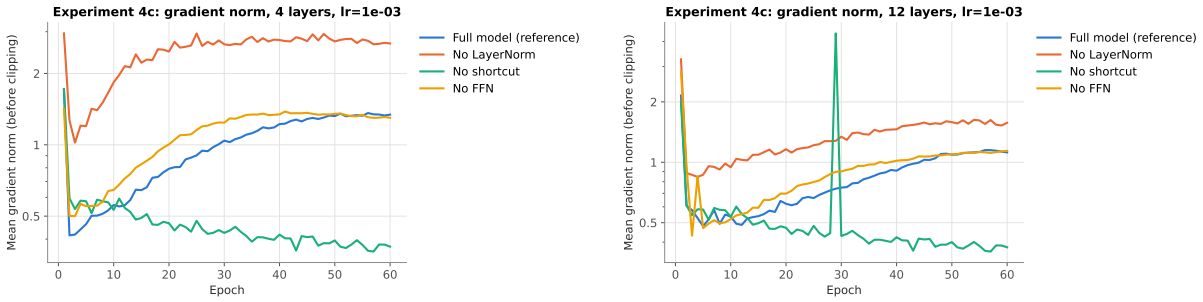
Figure 10: Ablations under the revised protocol (LR 10^{-3}, 60 epochs). Without residual connections (green) the loss stays at the unigram level of about 6.3.

- **The FFN** adds capacity. With one book there is too little to store, so its parameters did not improve validation loss.

9 Discussion

A data-limited regime. The model has about 172 parameters per training token, whereas compute-optimal training uses about 20 tokens per parameter [11]. Three observations show that data, not architecture, limits performance here. First, every model memorises the corpus: training loss approaches zero while validation loss rises. Second, all working architectures reach best validation losses in a narrow band (4.61–4.69 under the low learning rate). Third, the best transformer improves on a bigram counting model by only 0.33 nats (4.61 vs. 4.94), so most of what the model can extract from 168k tokens is local token statistics. We did not vary the dataset size, so this conclusion is an inference from these observations rather than a direct measurement; it is, however, consistent with scaling-law results [10, 11].

Capacity versus trainability. The ablations separate two kinds of components. *Capacity* components (depth, the FFN, more heads) increase what the model can represent; with little data they mostly accelerate memorisation, and none of them improved validation loss. *Trainability* components (residual connections, LayerNorm) determine whether optimisation succeeds at all. Their importance depends on depth and optimisation settings, not on data size: residual



(a) 4 layers **(b)** 12 layers

Figure 11: Mean gradient norm per epoch before clipping (revised protocol, logarithmic axis). Values above 1.0 are clipped to 1.0 before each update.

Table 11: Generated continuations of the prompt "I am sure that". Samples at epoch 60 come from the end of the revised protocol, not from the best epoch. ⋄ marks a partial byte token (half of a curly quotation mark) that cannot be displayed.

Model	Generated continuation (temperature 0.8, top-k 40, seed 123)
Baseline, epoch 1	*I am sure that* as as the„ very that. have. very ; I you ⋄„ ,I you was she. you, were,. that ⋄
Baseline, epoch 10	*I am sure that* and the other, for you have a great deal of you himself. I have had not you she_ did not so certain of my father to …
Baseline, epoch 20	*I am sure that* is not in the being so much disposed of not so much engaged. This I did not likely to be for the next morning at least …
LR 10^{-3}, epoch 500	*I am sure that* is but the only joined me with you. I had have you have done with an particulars." "I had my dear Miss Bingley; but I had …
Full model, 4L, ep. 60	*I am sure that* when we were very attentive to the very subject." "If he." "If he cried Mrs. JenkinsI am sorry with two or little my instrument," said Mrs. …
No LayerNorm, 4L, ep. 60	*I am sure that* theOh, my word is disposed to take it." Their first thought it of all; but he first witnessing his altered by Derbyshire. And yet seen such …
No shortcut, 4L, ep. 60	*I am sure that* as as the„. be the that., of have. . ; I you ,⋄„ , you was she. you was.,. it that is the
No FFN, 4L, ep. 60	*I am sure that* the become of her, and her, and her poultry. or affair is perfectly sure, and Jane may be attended to her, if he had never joined …

connections were indispensable even for 4 layers, while LayerNorm was unnecessary because the networks were shallow and gradient clipping bounded the updates. We expect LayerNorm to become necessary for deeper networks, higher learning rates or training without clipping, and capacity components to become useful with substantially more data.

Methodological lesson. The initial protocol, which compared models after a fixed number of epochs with a single seed, led to two conclusions that did not hold up: that the FFN helps, and (as a suspicion) that the LayerNorm result was an artefact. Comparing every model at its own best epoch removed the first, and confirmed the second as a real effect; repeating Experiments 2–3 over three seeds showed that the head effect is real while the depth differences are mostly noise. In a regime where every model overfits, the stopping point and the seed are confounders that must be controlled before architectural conclusions are drawn.

10 Limitations and Future Work

- **Single small corpus.** All results come from one novel (168k training tokens). The central claim about data limitation should be tested directly by training on several books (e.g. one versus five Austen novels) and checking whether depth and the FFN begin to help.
- **Scaled-down model.** Width 256 and context 256 instead of GPT-2's 768 and 1,024.
- **Simplified optimisation.** A constant learning rate without warm-up or decay, and gradient clipping in every run. Repeating the LayerNorm ablation without clipping and at larger depth would isolate the stabilising role of normalisation.
- **Seeds.** Experiments 2–3 use three seeds, Experiments 1 and 4 one seed. The large ablation effects (shortcut, LayerNorm) are far above the measured seed variation, but the FFN result is within it.
- **Partial normalisation ablation.** The final LayerNorm was kept in the no-LayerNorm variant.
- **Untested mechanism.** The explanation of the faster early learning without LayerNorm is a hypothesis; measuring the scale of each block's output during training would test it.
- **Text samples.** The samples of the revised protocol were taken at the end of training, not at the best epoch.

11 Conclusion

We trained a GPT-style model from scratch on one novel and varied training duration, learning rate, depth, attention heads and block components under a controlled protocol. The dominant effect is overfitting: every model reaches its best validation loss within 10–30 epochs and then memorises the training text, and a higher learning rate brings this point forward. In this data-limited regime, depth and the feed-forward network did not measurably improve generalisation, and fewer, wider attention heads were slightly better than many narrow ones. Residual connections were essential: without them the model learned nothing beyond token frequencies. LayerNorm was not needed and even reduced performance, because gradient clipping provided the necessary stability. Methodologically, the study shows that architecture comparisons in an overfitting regime must compare models at their own best epoch and over several seeds; a fixed-epoch, single-seed protocol gave misleading conclusions.

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[2] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. Language models are unsupervised multitask learners. OpenAI technical report, 2019.

[3] S. Raschka. *Build a Large Language Model (From Scratch).* Manning Publications, 2024.

[4] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[5] J. L. Ba, J. R. Kiros, and G. E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

[6] R. Xiong, Y. Yang, D. He, K. Zheng, S. Zheng, C. Xing, H. Zhang, Y. Lan, L. Wang, and T.-Y. Liu. On layer normalization in the transformer architecture. In *International Conference on Machine Learning (ICML)*, 2020.

[7] D. Hendrycks and K. Gimpel. Gaussian error linear units (GELUs). *arXiv preprint arXiv:1606.08415*, 2016.

[8] M. Geva, R. Schuster, J. Berant, and O. Levy. Transformer feed-forward layers are key-value memories. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2021.

[9] P. Michel, O. Levy, and G. Neubig. Are sixteen heads really better than one? In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[10] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

[11] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, et al. Training compute-optimal large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[12] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.

[13] R. Pascanu, T. Mikolov, and Y. Bengio. On the difficulty of training recurrent neural networks. In *International Conference on Machine Learning (ICML)*, 2013.

[14] R. Sennrich, B. Haddow, and A. Birch. Neural machine translation of rare words with subword units. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2016.

[15] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15:1929–1958, 2014.

A Code

The complete code, including the follow-up cells for the revised ablations and the seed runs, and the outputs of every run are available in the Google Colab notebook:

https://colab.research.google.com/drive/1zfz-atgfb0Z72t7IwxTM5HA-_C3_LFRG?usp=sharing

The notebook, this report's LaTeX source, all result tables and figures, and a script that reproduces the count-based reference models are also published in the GitHub repository https://github.com/Pratik2207/llm-from-scratch-ablation-study. The listings below show the parts that define the experiments.

Transformer block with ablation switches.

```
class TransformerBlock(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        d = cfg["emb_dim"]
        self.att = MultiHeadAttention(cfg)
        self.ff = FeedForward(cfg) if cfg["use_ffn"] else None
        self.norm1 = nn.LayerNorm(d) if cfg["use_norm"] else nn.Identity()
        self.norm2 = nn.LayerNorm(d) if cfg["use_norm"] and cfg["use_ffn"] else nn.Identity()
        self.drop = nn.Dropout(cfg["drop_rate"])
        self.use_shortcut = cfg["use_shortcut"]

    def forward(self, x):
        h = self.drop(self.att(self.norm1(x)))
        x = x + h if self.use_shortcut else h
        if self.ff is not None:
            h = self.drop(self.ff(self.norm2(x)))
            x = x + h if self.use_shortcut else h
        return x
```

One training step.

```
with torch.autocast(device_type="cuda", dtype=torch.float16):
    logits = model(X_train[b])
loss = F.cross_entropy(logits.float().flatten(0, 1), Y_train[b].flatten())
opt.zero_grad(set_to_none=True)
scaler.scale(loss).backward()
scaler.unscale_(opt)
gnorm = torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0) # logged per epoch
scaler.step(opt)
scaler.update()
```

Count-based reference models (Section 3.4).

```
tr, va = train_ids.tolist(), val_ids.tolist()
V, N = 50257, len(tr)
uni = collections.Counter(tr)
big = collections.Counter(zip(tr, tr[1:]))
prev = collections.Counter(tr[:-1])
p_uni = lambda t: (uni[t] + 0.1) / (N + 0.1 * V)
unigram_loss = -np.mean([math.log(p_uni(t)) for t in va]) # 6.32
lam = 0.7
bigram_loss = -np.mean([math.log(lam * (big[(a, b)] / prev[a] if prev[a] else 0)
                                 + (1 - lam) * p_uni(b))
                        for a, b in zip(va, va[1:])]) # 4.94
```

B Additional Figures

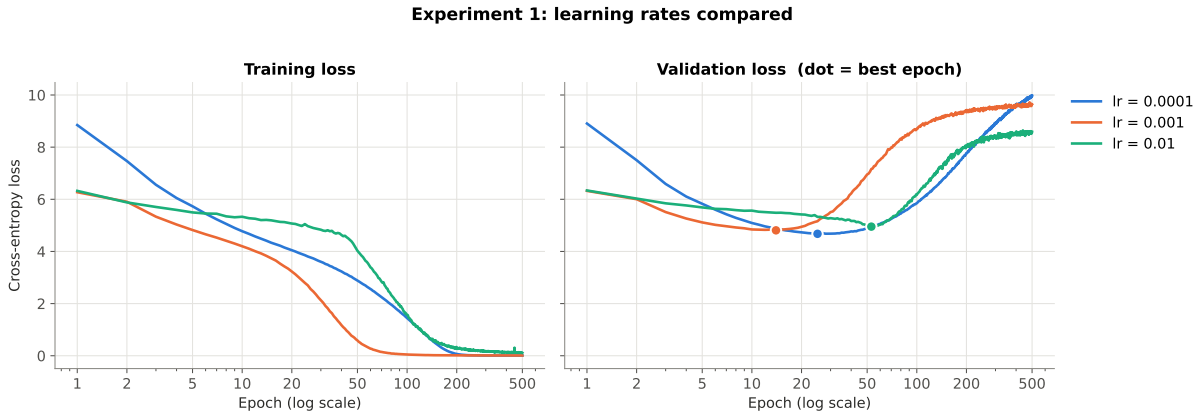


Figure 12: Experiment 1: all learning rates on common axes.

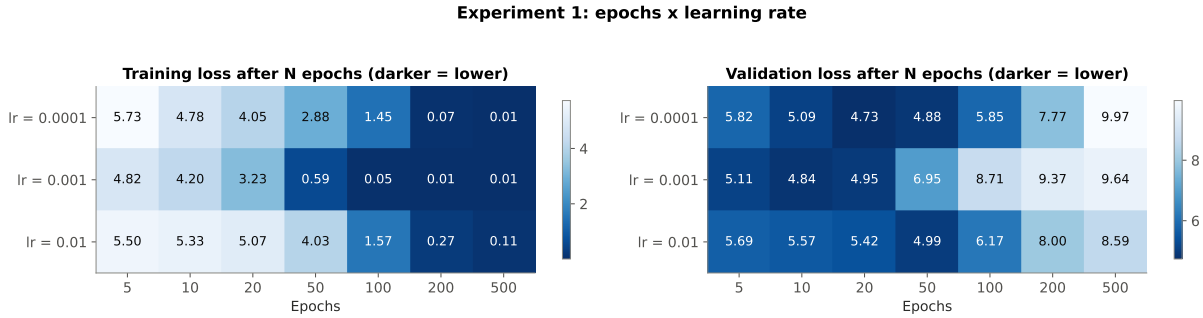


Figure 13: Experiment 1: training and validation loss after N epochs (darker is lower).

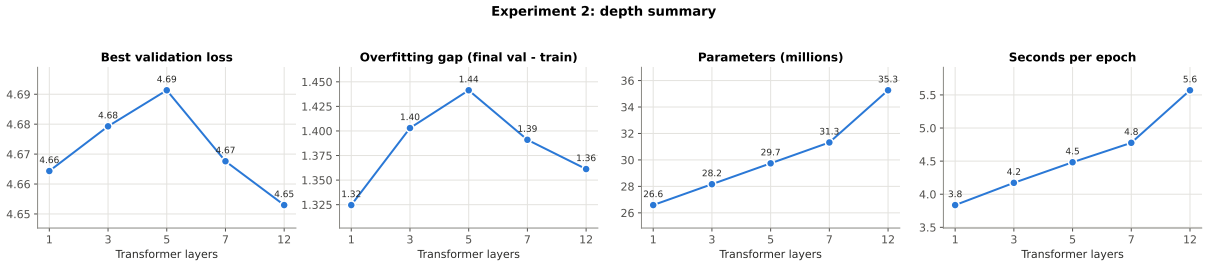


Figure 14: Experiment 2 (seed 123): best validation loss, final generalisation gap, parameters and seconds per epoch against depth.

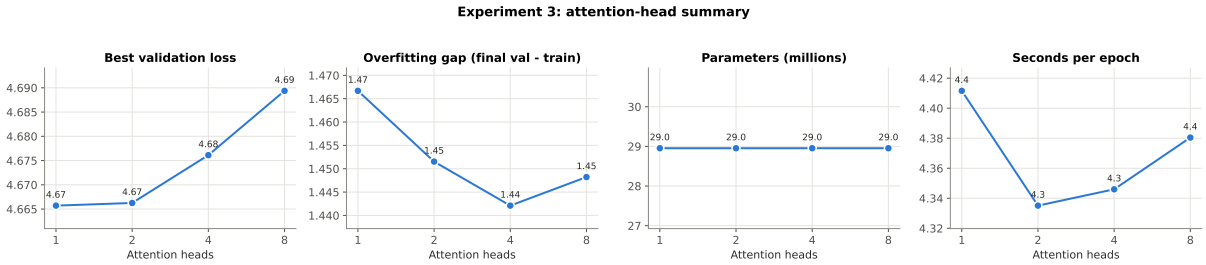
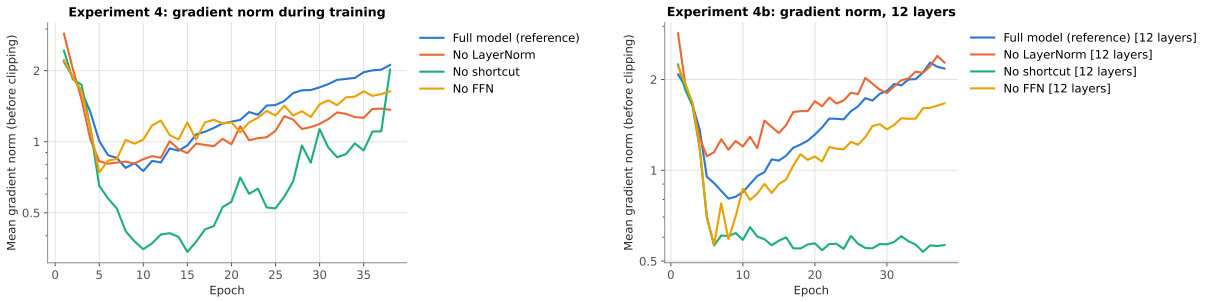
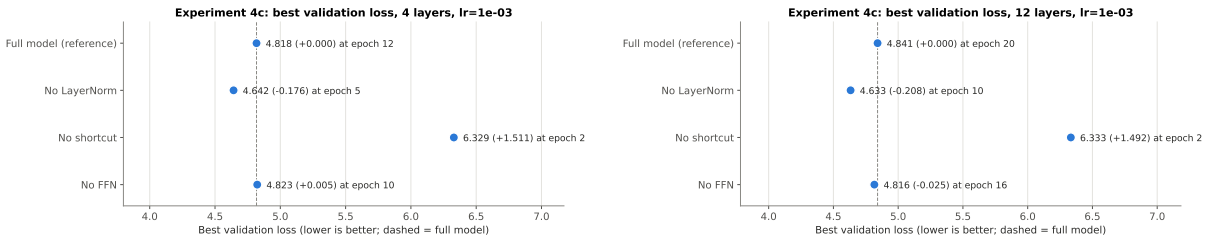


Figure 15: Experiment 3 (seed 123): the same summary against the number of heads.



(a) 4 layers **(b)** 12 layers

Figure 16: Gradient norm under the initial ablation protocol (LR 10^{-4}).



(a) 4 layers **(b)** 12 layers

Figure 17: Best validation loss of each ablation variant under the revised protocol (LR 10^{-3}).

C Complete Results

Table 12: All 47 training runs (Colab T4 GPU). L = layers, H = heads, Ep. = epochs trained, Min. = minutes per run.

Run	L	H	LR	Seed	Ep.	Params (M)	Best val.	Best ep.	Final train	Final val.	Min.
Baseline											
Baseline	4	4	4×10^{-4}	123	20	28.95	4.616	10	2.473	4.973	1.4
Experiment 1: epochs and learning rate											
LR 0.0001	4	4	1×10^{-4}	123	500	28.95	4.676	25	0.006	9.973	36.2
LR 0.001	4	4	1×10^{-3}	123	500	28.95	4.809	14	0.009	9.645	35.8
LR 0.01	4	4	1×10^{-2}	123	500	28.95	4.951	53	0.113	8.588	35.7
Experiment 2: layers (seed 123)											
1 layer	1	4	1×10^{-4}	123	38	26.59	4.664	30	3.369	4.694	2.4
3 layers	3	4	1×10^{-4}	123	38	28.16	4.679	25	3.337	4.740	2.6
5 layers	5	4	1×10^{-4}	123	38	29.74	4.691	30	3.311	4.753	2.8
7 layers	7	4	1×10^{-4}	123	38	31.32	4.668	29	3.308	4.699	3.0
12 layers	12	4	1×10^{-4}	123	38	35.27	4.653	29	3.335	4.696	3.5
Experiment 3: heads (seed 123)											
1 head	4	1	1×10^{-4}	123	38	28.95	4.666	29	3.271	4.738	2.8
2 heads	4	2	1×10^{-4}	123	38	28.95	4.666	29	3.278	4.730	2.7
4 heads	4	4	1×10^{-4}	123	38	28.95	4.676	25	3.299	4.741	2.8
8 heads	4	8	1×10^{-4}	123	38	28.95	4.689	25	3.306	4.754	2.8
Experiment 4: ablations, initial protocol											
Full model	4	4	1×10^{-4}	123	38	28.95	4.676	25	3.299	4.741	2.8
No LayerNorm	4	4	1×10^{-4}	123	38	28.95	4.627	37	3.710	4.630	2.7
No shortcut	4	4	1×10^{-4}	123	38	28.95	5.564	38	5.281	5.564	2.7
No FFN	4	4	1×10^{-4}	123	38	26.85	4.747	37	3.751	4.748	2.5
Full model	12	4	1×10^{-4}	123	38	35.27	4.653	29	3.335	4.696	3.5
No LayerNorm	12	4	1×10^{-4}	123	38	35.25	4.613	37	3.799	4.616	3.4
No shortcut	12	4	1×10^{-4}	123	38	35.27	6.316	11	6.274	6.342	3.4
No FFN	12	4	1×10^{-4}	123	38	28.95	4.824	37	4.047	4.831	2.9
Experiment 4: ablations, revised protocol											
Full model	4	4	1×10^{-3}	123	60	28.95	4.818	12	0.273	7.454	4.4
No LayerNorm	4	4	1×10^{-3}	123	60	28.95	4.642	5	0.403	7.476	4.3
No shortcut	4	4	1×10^{-3}	123	60	28.95	6.329	2	6.269	6.364	4.3
No FFN	4	4	1×10^{-3}	123	60	26.85	4.823	10	0.123	8.250	4.1
Full model	12	4	1×10^{-3}	123	60	35.27	4.841	20	0.320	7.171	5.6
No LayerNorm	12	4	1×10^{-3}	123	60	35.25	4.633	10	0.233	7.375	5.5
No shortcut	12	4	1×10^{-3}	123	60	35.27	6.333	2	6.269	6.363	5.5
No FFN	12	4	1×10^{-3}	123	60	28.95	4.816	16	0.536	7.340	4.7
Experiment 2: additional seeds											
1 layers	1	4	1×10^{-4}	42	38	26.59	4.658	32	3.385	4.675	2.5
1 layers	1	4	1×10^{-4}	7	38	26.59	4.666	30	3.370	4.706	2.5
3 layers	3	4	1×10^{-4}	42	38	28.16	4.669	30	3.328	4.711	2.7
3 layers	3	4	1×10^{-4}	7	38	28.16	4.659	31	3.329	4.716	2.7
5 layers	5	4	1×10^{-4}	42	38	29.74	4.682	28	3.305	4.733	2.9
5 layers	5	4	1×10^{-4}	7	38	29.74	4.664	27	3.310	4.717	2.9
7 layers	7	4	1×10^{-4}	42	38	31.32	4.655	27	3.306	4.699	3.1
7 layers	7	4	1×10^{-4}	7	38	31.32	4.662	27	3.311	4.714	3.1
12 layers	12	4	1×10^{-4}	42	38	35.27	4.659	31	3.316	4.701	3.6
12 layers	12	4	1×10^{-4}	7	38	35.27	4.643	27	3.330	4.681	3.6
Experiment 3: additional seeds											
1 heads	4	1	1×10^{-4}	42	38	28.95	4.663	28	3.298	4.720	2.8
1 heads	4	1	1×10^{-4}	7	38	28.95	4.661	28	3.274	4.713	2.9
2 heads	4	2	1×10^{-4}	42	38	28.95	4.670	27	3.300	4.717	2.8
2 heads	4	2	1×10^{-4}	7	38	28.95	4.663	28	3.301	4.704	2.8
4 heads	4	4	1×10^{-4}	42	38	28.95	4.677	28	3.299	4.722	2.8
4 heads	4	4	1×10^{-4}	7	38	28.95	4.682	27	3.312	4.724	2.8
8 heads	4	8	1×10^{-4}	42	38	28.95	4.678	27	3.309	4.736	2.8

Run	L	H	LR	Seed	Ep.	Params (M)	Best val.	Best ep.	Final train	Final val.	Min.
8 heads	4	8	1×10^{-4}	7	38	28.95	4.679	28	3.313	4.750	2.8